

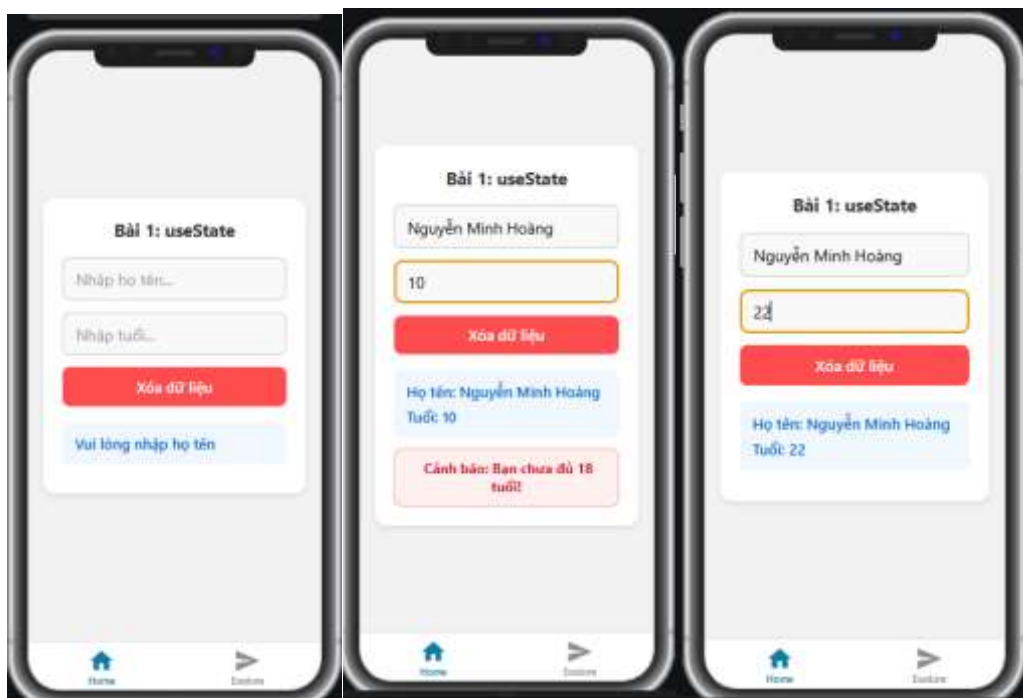
Tên: Nguyễn Minh Hoàng

MSSV: 22659081

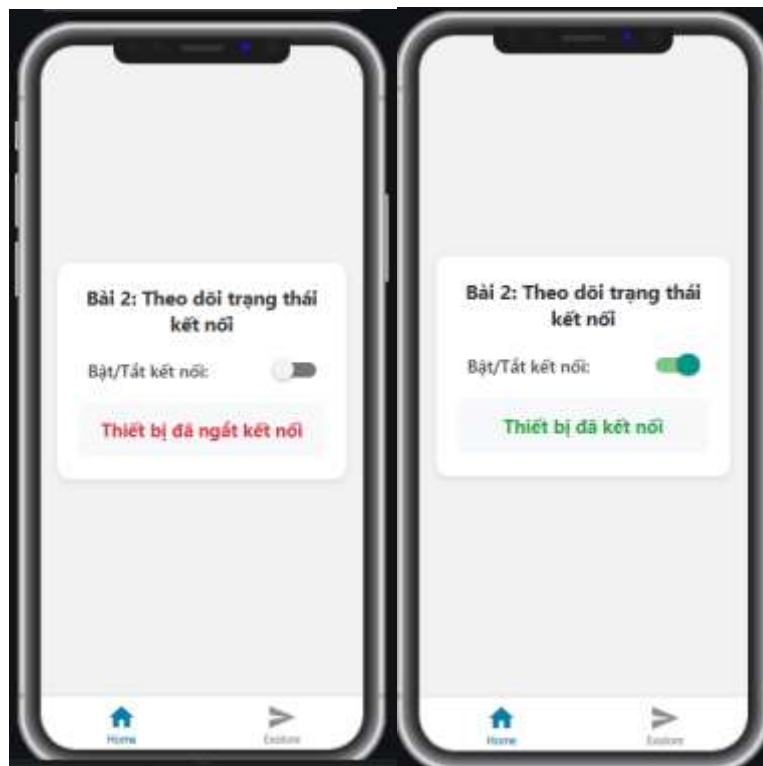
Github:

https://github.com/minhhoanghtm/Lab_Report_Mobile/tree/main/22659081_NguyenMinhHoang_Tuan03Nhom03

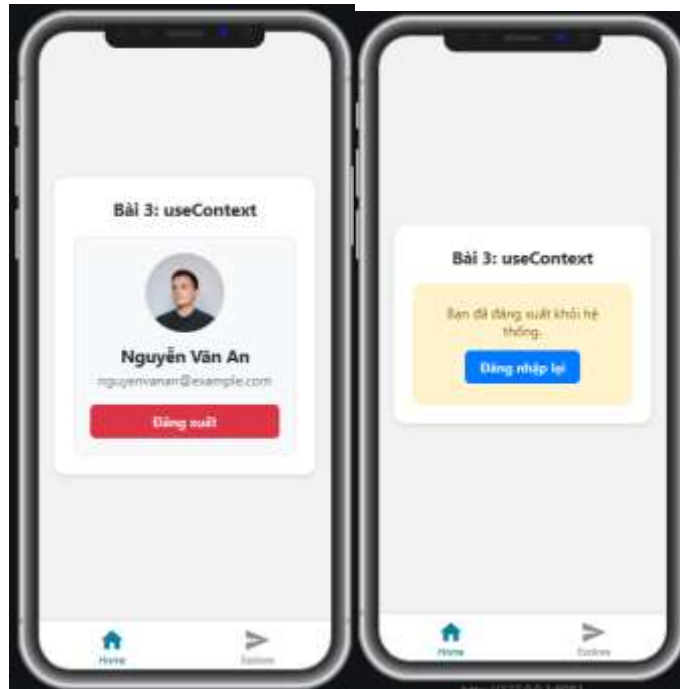
Bài 1: useState



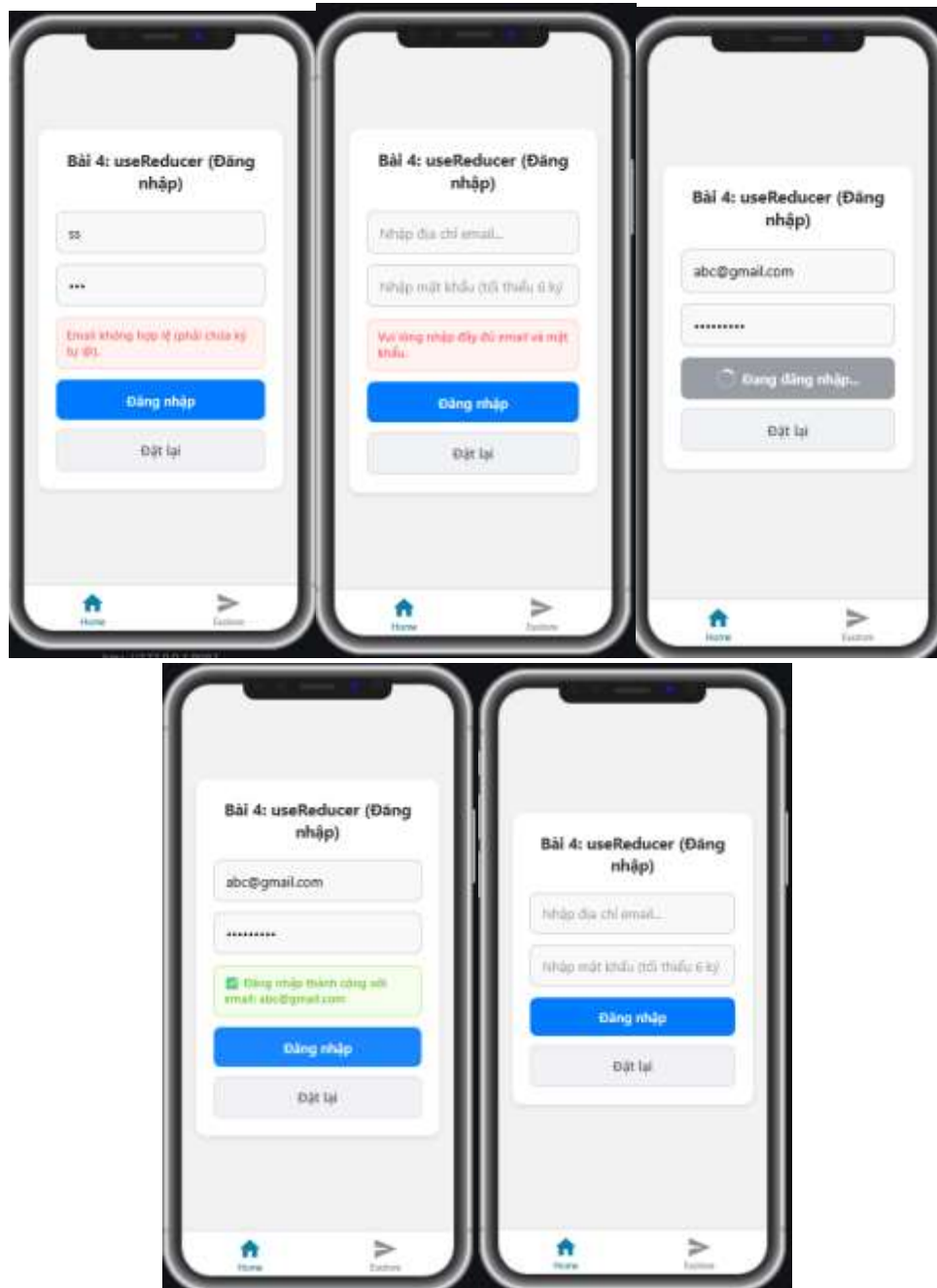
Bài 2: useEffect:



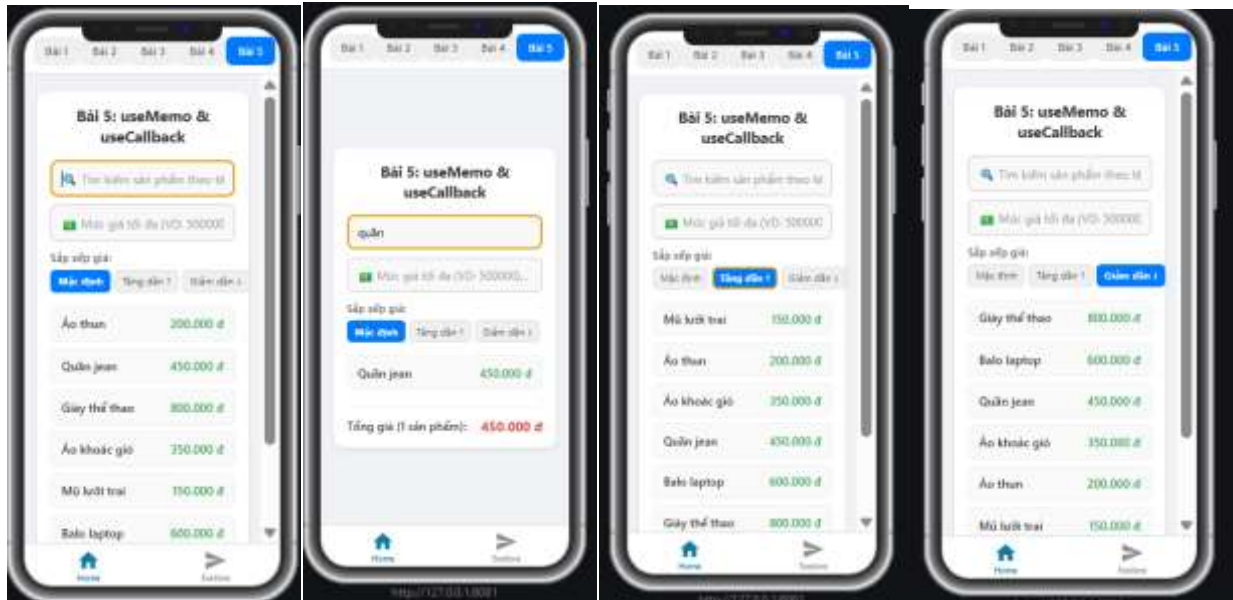
Bài 3: useContext



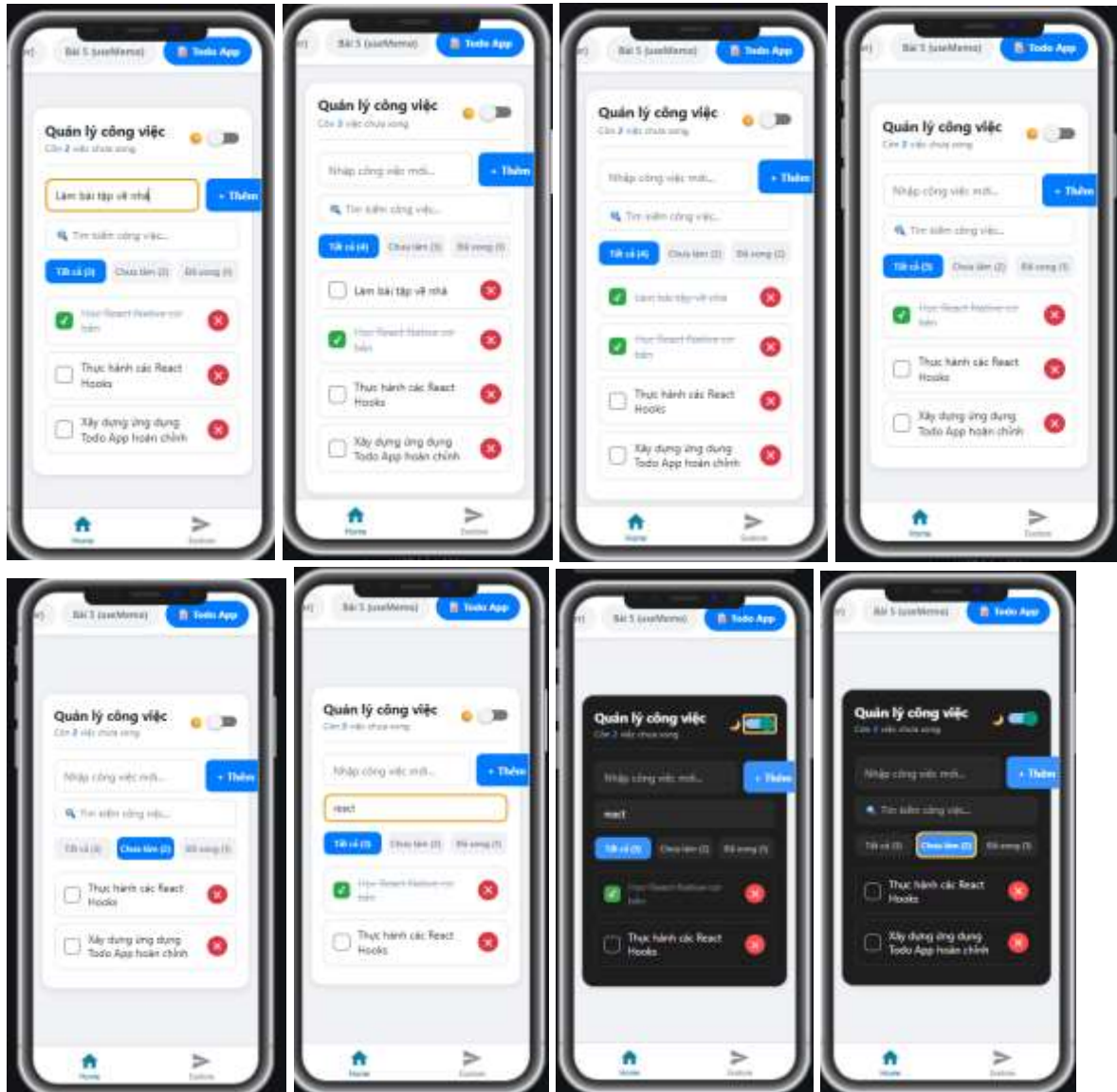
Bài 4: useReducer



Bài 5: UseMemo & useCallback



Bài tập tổng hợp cuối chương



Câu hỏi ôn tập

1. Khi nào nên dùng `useState` thay vì `useReducer`?

- Nên dùng **`useState`** khi state đơn giản và việc cập nhật state không có nhiều logic phức tạp. `useReducer` phù hợp hơn khi state phức tạp, có nhiều loại hành động hoặc nhiều state liên quan với nhau.

2. Dependency array của `useEffect` có tác dụng gì?

- Dependency array dùng để xác định **khi nào Effect được thực thi lại**. React sẽ theo dõi các giá trị trong dependency array và chạy lại Effect khi một trong các giá trị đó thay đổi.

3. Tại sao effect sử dụng bộ đếm thời gian cần có hàm dọn dẹp?

- Vì các bộ đếm thời gian như `setInterval` hoặc `setTimeout` có thể tiếp tục hoạt động sau khi component bị unmount hoặc Effect chạy lại. Hàm dọn dẹp giúp hủy timer, tránh rò rỉ bộ nhớ và tránh việc có nhiều timer chạy đồng thời không cần thiết.

4. `useContext` giải quyết hạn chế nào của việc truyền props?

- `useContext` giải quyết vấn đề **Prop Drilling**. Đây là tình trạng phải truyền dữ liệu qua nhiều component trung gian chỉ để đưa dữ liệu đến một component ở sâu bên trong cây component.

5. `useMemo` ghi nhớ giá trị hay hàm?

- `useMemo` ghi nhớ **giá trị**. Nó lưu lại kết quả của một phép tính và chỉ tính toán lại khi các dependency thay đổi.

6. `useCallback` ghi nhớ giá trị hay hàm?

- `useCallback` ghi nhớ **hàm**. Nó giúp giữ lại cùng một tham chiếu của hàm giữa các lần render và chỉ tạo hàm mới khi dependency thay đổi.

7. Vì sao không nên sử dụng `useMemo` và `useCallback` cho mọi trường hợp?

- Vì việc sử dụng chúng cũng tạo ra chi phí xử lý và làm code phức tạp hơn. Chúng chỉ nên được sử dụng khi thực sự cần tối ưu hiệu năng. Nếu áp dụng không cần thiết, lợi ích có thể không đáng kể và thậm chí làm ứng dụng khó bảo trì hơn.

8. Reducer có được thay đổi trực tiếp state hiện tại không? Vì sao?

- Reducer **không nên thay đổi trực tiếp state hiện tại**. Thay vào đó, reducer cần tạo và trả về một state mới. Điều này giúp đảm bảo tính bất biến của state, giúp React nhận biết sự thay đổi một cách chính xác và làm cho việc quản lý, dự đoán và kiểm tra state dễ dàng hơn.